FPGA Workshop 3

Sponsored by Jane Street

Contents

1. RECAP + small yap
2. Debouncing
3. FSM in FGPA
4. Multiplexing
5. Find a team!

1. Recap

Who did the homework?

Show us your solutions!

1. Answer / Bugs

In `counter_example`, you have:

```
if (count == 50_000_000) count <= 0;
```

If your clock is exactly 50MHz, it ticks 50,000,000 times in one second. However, since you are counting from **0** to **50,000,000**, you are actually waiting for **50,000,001** clock cycles.

- **The Fix:** Use `if (count == 49_999_999)`. This ensures that 0 through 49,999,999 covers exactly 50 million cycles.

In `tb_counter`, you didn't actually instantiate your `counter_example` module. Instead, you copy-pasted the logic into the testbench:

- **The Issue:** This is a "lab foul." You are testing your testbench's ability to count, not your actual hardware module. If you fix a bug in `counter_example.v`, your testbench won't show the fix because it's using its own separate logic.
- **The Fix:** Instantiate it like you did in `tb_top` using `counter_example dut (...)`.

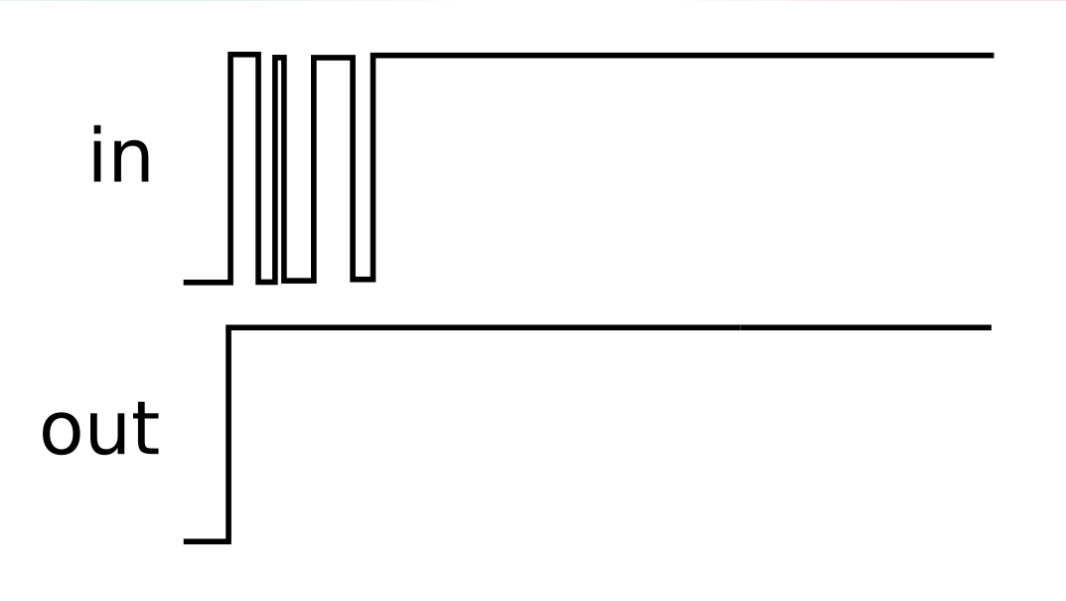
2. Debouncing

2. Debouncing

When you press a mechanical button or flip a switch, you might think the electrical connection goes from 0 to 1 instantly. In reality, buttons are made of springy metal contacts.

When those contacts meet, they don't just stay together immediately; they **bounce** off each other for a few milliseconds before settling into a steady state. This is called "Contact Bounce."

2. Debouncing



2. Debouncing

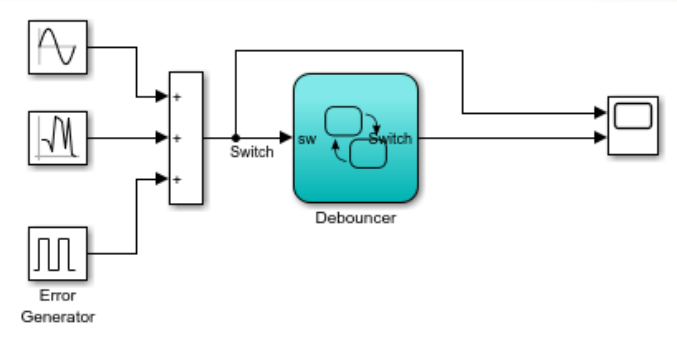
The reason this is a "problem" specifically for FPGAs is the massive difference in speed between a human and a chip:

- **The Human:** A "quick" button press takes about 100 milliseconds.
- **The Bounce:** The mechanical vibrating (bouncing) usually lasts between **5ms to 10ms**.
- **The FPGA:** Your 50MHz clock "ticks" every **20 nanoseconds**.

To a 50MHz FPGA, those 10 milliseconds of bouncing look like an eternity. Instead of seeing one "Press," the FPGA sees **hundreds of thousands of tiny on-off pulses**.

2. Debouncing

The Result: If you have a stopwatch and you press "Pause," the FPGA might see 50 "presses" in the span of a millisecond. The stopwatch will pause, unpause, pause, unpause... and you'll end up with a random result.



2. Debouncing

- To fix this in Verilog, you create a "Digital Filter." The logic is simple: **Wait and See.**
- **Detection:** The moment you see the button signal change (from 0 to 1), you start a counter.
- **Validation:** You ignore any further changes for a set period (usually **20ms**).
- **Final Check:** After the 20ms is up, you check the button again.
  - If it is *still* 1, you consider it a "Valid Press" and send a signal to the rest of your system.
  - If it went back to 0, it was probably just electrical noise or a "glitch."

2. Debouncing

Why do we need it?

Whether it's a button press on a desk or a data packet arriving from a fiber optic cable, engineers have to account for "noise" and ensure the system only reacts to stable, valid signals.

We can’t trust external inputs at face value.

2. Debouncing

Let’s start by declaring a new module.

Inputs and Outputs.

```
module debouncer (
   input clk,
   input btn_in,
   output reg btn_out
);
   reg [19:0] count;
   reg sync_0, sync_1;

   always @(posedge clk) begin
      sync_0 <= btn_in;
      sync_1 <= sync_0;

      if (sync_1 != btn_out) begin
         count <= count + 1;
         if (count == 20'd1_000_000) begin
            btn_out <= sync_1;
            count <= 0;
         end
      end else begin
         count <= 0;
      end
   end
endmodule
```

2. Debouncing

Next, we'll set a 20-bit register for counting.

20 bits lets us count to ~1,000,000 (20ms)

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 <= btn_in;
    sync_1 <= sync_0;

    if (sync_1 != btn_out) begin
      count <= count + 1;
      if (count == 20'd1_000_000) begin
        btn_out <= sync_1;
        count <= 0;
      end
    end else begin
      count <= 0;
    end
  end
endmodule
```

2. Debouncing

- Imagine the clock is a camera shutter and the button signal is a person jumping. If the person jumps at the *exact same nanosecond* the shutter clicks, the photo might be a blurry mess.

- In silicon, if the button signal changes state at the exact same time the clock edge arrives, the Flip-Flop (the reg) gets "confused." It enters a state called **Metastability**, where the voltage is neither a 0 nor a 1: it's floating somewhere in the middle.

- If that "blurry" signal travels to the rest of your project, it can cause the whole chip to glitch or crash.

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 <= btn_in;
    sync_1 <= sync_0;

    if (sync_1 != btn_out) begin
      count <= count + 1;
      if (count == 20'd1_000_000) begin
        btn_out <= sync_1;
        count <= 0;
      end
    end else begin
      count <= 0;
    end
  end
endmodule
```

2. Debouncing

- By writing **reg sync_0, sync_1**; and using them in a chain, you are building a Synchronizer.
- **sync_0 (The Buffer):** This register takes the direct hit from the button. If the button signal is "blurry" (metastable), sync_0 might become blurry too.
- **The Wait**: The signal now has one full clock cycle (20 nanoseconds) to "settle down" into a solid 0 or 1.

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 < = btn_in;
    sync_1 < = sync_0;

    if (sync_1 != btn_out) begin
      count < = count + 1;
      if (count == 20'd1_000_000) begin
        btn_out < = sync_1;
        count < = 0;
      end
    end else begin
      count < 0;
    end
  end
endmodule
```

2. Debouncing

- **if (sync_1 != btn_out) sync_1**: This is the current (tamed) state of the physical button right now.
- **btn**_out: This is the "Stable" state we currently believe in (the one we send to the rest of the project).
- If they are different, it means the button is either being pressed, being released, or is currently "bouncing."
- The FPGA notices a discrepancy and says, "Something is happening, let's start timing it."

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 < = btn_in;
    sync_1 < = sync_0;

    if (sync_1 != btn_out) begin
      count < = count + 1;
      if (count == 20'd1_000_000) begin
        btn_out < = sync_1;
        count < = 0;
      end
    end else begin
      count < 0;
    end
  end
endmodule
```

2. Debouncing

- As long as the physical button (sync_1) is different from our stable state (btn_out), we increment the counter.
- We are measuring duration.
- We want to see how long the button can hold its new position.

```
module debouncer (
   input clk,
   input btn_in,
   output reg btn_out
);
   reg [19:0] count;
   reg sync_0, sync_1;

   always @(posedge clk) begin
      sync_0 < = btn_in;
      sync_1 < = sync_0;

      if (sync_1 != btn_out) begin
         count < = count + 1;
         if (count == 20'd1_000_000) begin
            btn_out < = sync_1;
            count < = 0;
         end
      end else begin
         count < 0;
      end
   end
endmodule
```

2. Debouncing

- At 50MHz, 1,000,000 clock cycles is exactly 20 milliseconds. Why 20ms? Most mechanical buttons finish bouncing within 5-10ms.
- By waiting 20ms, we are being 100% sure the vibration has stopped.
- Once the counter hits this "Confidence Threshold," we finally update btn_out <= sync_1.
- The rest of the project now sees the new button state.

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 < = btn_in;
    sync_1 < = sync_0;

    if (sync_1 != btn_out) begin
      count < = count + 1;
      if (count == 20'd1_000_000) begin
        btn_out < = sync_1;
        count < = 0;
      end
    end else begin
      count < = 0;
    end
  end
endmodule
```

2. Debouncing

- This is the most important part of the filter. If the button is bouncing (flicking between 0 and 1), it will occasionally match btn_out for a split second.
- When that happens, the else kicks in and resets the counter to zero.
- This means the "20ms timer" only finishes if the button is perfectly still for the entire duration.
- If it jitters even once, the timer starts over.

```
module debouncer (
  input clk,
  input btn_in,
  output reg btn_out
);
  reg [19:0] count;
  reg sync_0, sync_1;

  always @(posedge clk) begin
    sync_0 < = btn_in;
    sync_1 < = sync_0;

    if (sync_1 != btn_out) begin
      count < = count + 1;
      if (count == 20'd1_000_000) begin
        btn_out < = sync_1;
        count < = 0;
      end
    end else begin
      count < = 0;
    end
  end
endmodule
```

2. Debouncing

Put it all together:

- A person (the button signal) arrives and tries to get in.
- The doorman doesn't let them in immediately. He points at a stopwatch and says, "**Stand perfectly still for 20 seconds**."
- If the person wobbles or flinches (a bounce), the doorman resets the watch to zero and says, "Start over."
- Only when the person stands perfectly still for the full time does the doorman open the door (btn_**out**).

```
module debouncer (
   input clk,
   input btn_in,
   output reg btn_out
);
   reg [19:0] count;
   reg sync_0, sync_1;

   always @(posedge clk) begin
      sync_0 <= btn_in;
      sync_1 <= sync_0;

      if (sync_1 != btn_out) begin
         count <= count + 1;
         if (count == 20'd1_000_000) begin
            btn_out <= sync_1;
            count <= 0;
         end
      end else begin
         count <= 0;
      end
   end
endmodule
```

2. Debouncing

Now try it on your PC, use the testbench

debouncer_tb.v

```
module debouncer (
   input clk,
   input btn_in,
   output reg btn_out
);
   reg [19:0] count;
   reg sync_0, sync_1;

   always @(posedge clk) begin
      sync_0 <= btn_in;
      sync_1 <= sync_0;

      if (sync_1 != btn_out) begin
         count <= count + 1;
         if (count == 20'd1_000_000) begin
            btn_out <= sync_1;
            count <= 0;
         end
      end else begin
         count <= 0;
      end
   end
endmodule
```

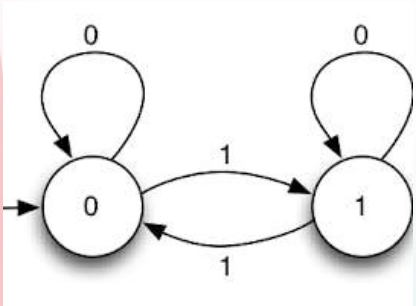
3. FSM

2. FSM

- Finite State Machines (FSMs) are the most powerful way to organize hardware. In software, you have "functions" and "loops." In hardware, you have **States**.
- Think of an FSM as a map. Each bubble is a state, and each arrow is a condition (like a button press).

2. FSM

- Think of an FSM as a map. Each bubble is a state, and each arrow is a condition (like a button press).



2. FSM: 3 Block Style

- **A way of writing Verilog where we** separate the logic so it’s easy to debug.
- **State Definitions:** Name your states so you don't have to remember binary numbers.
- **State Register (Sequential):** The memory that holds where we are.
- **Next State Logic (Combinational):** The "brain" that looks at the map and decides where to go next.

3. FSM: 3 Block

- Declare the inputs and outputs
- **btn_start_stop** - use the debounced version
- **run_en** – signal to the counter to stop/start

1/2

```
module stopwatch_fsm (
    input clk,
    input rst,
    input btn_start_stop,
    output reg run_en
);

    // 1. State Definitions
    localparam S_IDLE    = 2'b00;
    localparam S_RUNNING = 2'b01;
    localparam S_PAUSED  = 2'b10;

    reg [1:0] current_state, next_state;

    // 2. State Register (Sequential - "The Heartbeat")
    always @(posedge clk) begin
        if (rst)
            current_state <= S_IDLE;
        else
            current_state <= next_state;
    end
```

```
module stopwatch_fsm (
    input clk,
    input rst,
    input btn_start_stop,
    output reg run_en
);

    // 1. State Definitions
    localparam S_IDLE    = 2'b00;
    localparam S_RUNNING = 2'b01;
    localparam S_PAUSED  = 2'b10;

    reg [1:0] current_state, next_state;

    // 2. State Register (Sequential - "The Heartbeat")
    always @(posedge clk) begin
        if (rst)
            current_state <= S_IDLE;
        else
            current_state <= next_state;
    end
```

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- Declare labels for the states.
- We’re using **localparam** to do this.
- These are constants to make it human readable
- **cannot be overridden** when the module is instantiated
- Configures state machine numbers in binary


```
module stopwatch_fsm (
    input clk,
    input rst,
    input btn_start_stop,
    output reg run_en
);

    // 1. State Definitions
    localparam S_IDLE    = 2'b00;
    localparam S_RUNNING = 2'b01;
    localparam S_PAUSED  = 2'b10;

    reg [1:0] current_state, next_state;

    // 2. State Register (Sequential - "The Heartbeat")
    always @(posedge clk) begin
        if (rst)
            current_state <= S_IDLE;
        else
            current_state <= next_state;
    end
```

3. FSM: 3 Block

- We need two registers. One to hold where we are **now**, and one to calculate where we are going **next**.
- We can declare in one line.
- These are two bit registers as states are two binary digits long.


```
module stopwatch_fsm (
    input clk,
    input rst,
    input btn_start_stop,
    output reg run_en
);

    // 1. State Definitions
    localparam S_IDLE    = 2'b00;
    localparam S_RUNNING = 2'b01;
    localparam S_PAUSED  = 2'b10;

    reg [1:0] current_state, next_state;

    // 2. State Register (Sequential - "The Heartbeat")
    always @(posedge clk) begin
        if (rst)
            current_state <= S_IDLE;
        else
            current_state <= next_state;
    end
```

3. FSM: 3 Block

- **always @(posedge clk)** - This logic only wakes up at the exact moment the clock "ticks" (the rising edge).
- **if (rst):** Hardware doesn't start at zero automatically. This line ensures that if the reset button is hit, the "brain" returns to the start (S_IDLE).
- **current_state <= next_state**: This is the actual "movement." At every tick, the FSM "steps" into the state we calculated in the next block.
- The **<=** (Non-blocking) We use non-blocking because this is **Sequential Logic** (Memory). We want all registers in the whole FPGA to update at the exact same moment.


```
module stopwatch_fsm (
    input clk,
    input rst,
    input btn_start_stop,
    output reg run_en
);

    // 1. State Definitions
    localparam S_IDLE    = 2'b00;
    localparam S_RUNNING = 2'b01;
    localparam S_PAUSED  = 2'b10;

    reg [1:0] current_state, next_state;

    // 2. State Register (Sequential - "The Heartbeat")
    always @(posedge clk) begin
        if (rst)
            current_state <= S_IDLE;
        else
            current_state <= next_state;
    end
```

3. FSM: 3 Block

- This is the logic for “next state” mappings

- **always @(*):** This block is "always awake." It doesn't wait for a clock tick. It is constantly calculating the next move based on current inputs.

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- '**case**' – hopefully obvious that it's a switch depending on what state we're currently in on the always call
- This is like a "You are here" marker on a map.
- The FPGA looks at which bubble it is currently in and ignores the logic for all other bubbles.

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- Now we'll look at each state individually
- IDLE:
- **run_en = 0**: In this bubble, the output to the counter is disabled
- "If I am sitting still and someone presses the button, move the map to the Running state."

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- Now we'll look at each state individually
- RUNNING:
- **run_en = 1:** This signal goes to your counter module to tell it "Go!"
- "If I am currently running and the button is pressed, move to the Paused state."

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- Now we'll look at each state individually
- PAUSED:
- This is different from IDLE because the counter doesn't reset to zero; it just stays where it is because run_en is now 0.

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- Now we'll look at each state individually
- DEFAULT:
- If a cosmic ray or electrical noise somehow flips a bit and the FSM ends up in a "ghost state" (like 2'b11, which we didn't define), this line teleports the FSM back to IDLE safely.

2/2

```
// 3. Next State Logic (Combinational - "The Map")
  // This block decides what next_state SHOULD be
  always @(*) begin
    case (current_state)
      S_IDLE: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_IDLE;
      end

      S_RUNNING: begin
        run_en = 1;
        if (btn_start_stop) next_state = S_PAUSED;
        else next_state = S_RUNNING;
      end

      S_PAUSED: begin
        run_en = 0;
        if (btn_start_stop) next_state = S_RUNNING;
        else next_state = S_PAUSED;
      end
      default: next_state = S_IDLE;
    endcase
  end
endmodule
```

3. FSM: 3 Block

- This module acts as a "control center" that manages the operational mode of the stopwatch by transitioning between three distinct states based on a single button input.
- It uses a sequential block to update the current state on every clock tick and a combinational block to define the logic for shifting between IDLE (reset/stopped), RUNNING (timer active), and PAUSED (timer held).
- When the state is RUNNING, the module outputs a high run_en signal, which serves as a master enable for the actual counter logic elsewhere in the system, ensuring the stopwatch only increments when the "brain" says it is in the correct mode.

- **Now try it with the testbench on Github**

4. Multiplexing

4. Multiplexing

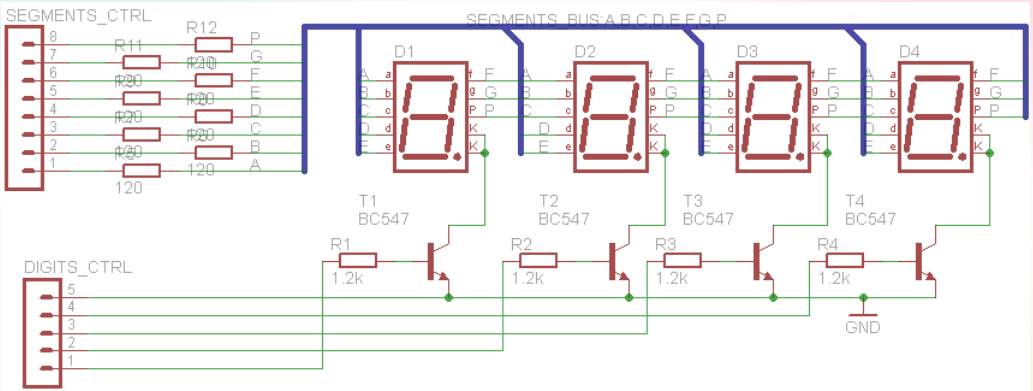
- **The Problem:**
- Pin Shortage: A 7-segment display has 7 segments + 1 decimal point = 8 pins.
- If you want a 4-digit display, you would need 4x8 = 32 pins just for the LEDs!
- The Reality: To save space and cost, FPGA boards connect the same 8 segment wires to every digit.
- The Conflict: If you send the signal for "5," all digits show "5.“

- **How do we solve this? Multiplexing.**

4. Multiplexing

- We don't show all the numbers at once. We show them one by one, very, very fast.
- **Step 1:** Turn on Digit 1 → Send "1" to the segments → Wait 1ms.
- **Step 2:** Turn off Digit 1 → Turn on Digit 2 → Send "2" to the segments → Wait 1ms.
- **Step 3:** Repeat!

4. Multiplexing



4. Multiplexing

- How does this actually work?
- The human eye cannot perceive flickering faster than about 60Hz to 100Hz.
- If we switch between digits at 1000Hz (1kHz), the brain "blends" the images together.
- You see a solid, multi-digit number like "12" instead of a flickering mess.
- We use only 10 pins (8 segments + 2 digit enables) instead of 16 pins to drive two digits. **EFFICIENT AF.**

"Multiplexing is like a movie or a flipbook. A movie is just 24 still pictures per second flashed in front of you. Because it's fast, your brain sees one continuous motion. In hardware, we flash '1' and then '2' so fast that your brain sees '12'."

4. Multiplexing

- How does this actually work?
- The human eye cannot perceive flickering faster than about 60Hz to 100Hz.
- If we switch between digits at 1000Hz (1kHz), the brain "blends" the images together.
- You see a solid, multi-digit number like "12" instead of a flickering mess.
- We use only 10 pins (8 segments + 2 digit enables) instead of 16 pins to drive two digits. **EFFICIENT AF**

4. Multiplexing

- Declare our inputs and outputs

```
module simple_mux (
  input clk,          // 50MHz
  input [3:0] tens,     // The number for the left digit
  input [3:0] units,    // The number for the right digit
  output reg [3:0] hex_to_display, // This goes to your hex_to_7seg decoder
  output reg [1:0] digit_select   // These go to the 'Anode' pins of the display
);

  // 1. Create a 1kHz switching signal (flicker_clk)
  reg [15:0] count;
  reg flicker_clk;

  always @(posedge clk) begin
    if (count == 16'd25_000) begin // Switch every 0.5ms (1kHz total)
      flicker_clk <= ~flicker_clk;
      count <= 0;
    end else begin
      count <= count + 1;
    end
  end

  // 2. The Multiplexer Logic
  always @(*) begin
    if (flicker_clk == 0) begin
      hex_to_display = units;   // Send units to the decoder
      digit_select = 2'b10;     // Turn on only the right digit (Active Low)
    end else begin
      hex_to_display = tens;    // Send tens to the decoder
      digit_select = 2'b01;     // Turn on only the left digit (Active Low)
    end
  end
endmodule
```

4. Multiplexing

- **reg [15:0] count**: A 16-bit register to hold our timing count.
- **if (count == 16'd25_000):** Since the clock is 50MHz (50 million ticks per second), counting to 25,000 takes exactly 0.5 milliseconds.
- **flicker_clk <= ~flicker_clk:** Every 0.5ms, we flip this bit. This creates a square wave that stays at 0 for 0.5ms and 1 for 0.5ms, completing a full cycle in 1ms (1kHz).
- **count <= 0:** We reset the timer to start the next 0.5ms window.

```
module simple_mux (
  input clk,          // 50MHz
  input [3:0] tens,     // The number for the left digit
  input [3:0] units,    // The number for the right digit
  output reg [3:0] hex_to_display, // This goes to your hex_to_7seg decoder
  output reg [1:0] digit_select   // These go to the 'Anode' pins of the display
);

  // 1. Create a 1kHz switching signal (flicker_clk)
  reg [15:0] count;
  reg flicker_clk;

  always @(posedge clk) begin
    if (count == 16'd25_000) begin // Switch every 0.5ms (1kHz total)
      flicker_clk <= ~flicker_clk;
      count <= 0;
    end else begin
      count <= count + 1;
    end
  end

  // 2. The Multiplexer Logic
  always @(*) begin
    if (flicker_clk == 0) begin
      hex_to_display = units;   // Send units to the decoder
      digit_select = 2'b10;     // Turn on only the right digit (Active Low)
    end else begin
      hex_to_display = tens;    // Send tens to the decoder
      digit_select = 2'b01;     // Turn on only the left digit (Active Low)
    end
  end
endmodule
```

4. Multiplexing

- **always @(*):** This is a Combinational block. It reacts instantly whenever flicker_clk changes.
- **hex_to_display = units:** When the flicker clock is low, we route the "Units" number (the 1s place) to our 7-segment decoder.
- **digit_select = 2'b10:** This is Active Low logic. 0 means ON and 1 means OFF. This turns the Right Digit ON and the Left Digit OFF.
- **else ... tens ... 2'b01:** 0.5ms later, we swap. We show the "Tens" number and turn ON only the Left Digit.

```
module simple_mux (
  input clk,          // 50MHz
  input [3:0] tens,     // The number for the left digit
  input [3:0] units,    // The number for the right digit
  output reg [3:0] hex_to_display, // This goes to your hex_to_7seg decoder
  output reg [1:0] digit_select   // These go to the 'Anode' pins of the display
);

  // 1. Create a 1kHz switching signal (flicker_clk)
  reg [15:0] count;
  reg flicker_clk;

  always @(posedge clk) begin
    if (count == 16'd25_000) begin // Switch every 0.5ms (1kHz total)
      flicker_clk <= ~flicker_clk;
      count <= 0;
    end else begin
      count <= count + 1;
    end
  end

  // 2. The Multiplexer Logic
  always @(*) begin
    if (flicker_clk == 0) begin
      hex_to_display = units;   // Send units to the decoder
      digit_select = 2'b10;     // Turn on only the right digit (Active Low)
    end else begin
      hex_to_display = tens;    // Send tens to the decoder
      digit_select = 2'b01;     // Turn on only the left digit (Active Low)
    end
  end
endmodule
```

4. Multiplexing

- **Overall:**
- The simple_mux acts like a high-speed physical switch. Instead of using 14 wires to drive two digits, it shares the same 7 wires for both digits.
- By toggling between the Tens and Units data 1,000 times per second, it exploits "Persistence of Vision."
- The hardware is only ever showing one digit at a time, but it switches so fast that the human brain merges them into a single, two-digit number.
- **We save physical pins on the chip by trading off speed (switching very fast).**

```
module simple_mux (
  input clk,          // 50MHz
  input [3:0] tens,     // The number for the left digit
  input [3:0] units,    // The number for the right digit
  output reg [3:0] hex_to_display, // This goes to your hex_to_7seg decoder
  output reg [1:0] digit_select   // These go to the 'Anode' pins of the display
);

  // 1. Create a 1kHz switching signal (flicker_clk)
  reg [15:0] count;
  reg flicker_clk;

  always @(posedge clk) begin
    if (count == 16'd25_000) begin // Switch every 0.5ms (1kHz total)
      flicker_clk <= ~flicker_clk;
      count <= 0;
    end else begin
      count <= count + 1;
    end
  end

  // 2. The Multiplexer Logic
  always @(*) begin
    if (flicker_clk == 0) begin
      hex_to_display = units;   // Send units to the decoder
      digit_select = 2'b10;     // Turn on only the right digit (Active Low)
    end else begin
      hex_to_display = tens;    // Send tens to the decoder
      digit_select = 2'b01;     // Turn on only the left digit (Active Low)
    end
  end
endmodule
```

4. Multiplexing

- **Now try it with the testbench!**

```
module simple_mux (
  input clk,          // 50MHz
  input [3:0] tens,     // The number for the left digit
  input [3:0] units,    // The number for the right digit
  output reg [3:0] hex_to_display, // This goes to your hex_to_7seg decoder
  output reg [1:0] digit_select   // These go to the 'Anode' pins of the display
);

  // 1. Create a 1kHz switching signal (flicker_clk)
  reg [15:0] count;
  reg flicker_clk;

  always @(posedge clk) begin
    if (count == 16'd25_000) begin // Switch every 0.5ms (1kHz total)
      flicker_clk <= ~flicker_clk;
      count <= 0;
    end else begin
      count <= count + 1;
    end
  end

  // 2. The Multiplexer Logic
  always @(*) begin
    if (flicker_clk == 0) begin
      hex_to_display = units;   // Send units to the decoder
      digit_select = 2'b10;     // Turn on only the right digit (Active Low)
    end else begin
      hex_to_display = tens;    // Send tens to the decoder
      digit_select = 2'b01;     // Turn on only the left digit (Active Low)
    end
  end
endmodule
```

5. Teams and Projects

5. Teams and Projects

- **Find Teams of 5! – This week**
- Work on a project for next three weeks
- We'll order Pynq Z1/Z2 for you to work on
- Best project gets made on a tinytapeout ASIC
- More details next week